

```

* 지정된 Key가 없으면 Character제공 특수문자 체크하고 Key가 있으면 해당 키를 체크
* </pre>
* @param str 문자 조합
* @param keys 문자 키 (default key = "!,*&^%$#@~;>+")
* @return boolean
*/
public static boolean isSpecialKey(String strVal, String keysVal) {
    String str = strVal;
    String keys = keysVal;
    boolean bReturn = false;

    if (keys == null) {
        char cCharAt = '\0';
        if (null != str) {
            int strLength = str.length();
            for (int i = 0; i < strLength; i++) {
                cCharAt = str.charAt(i);
                if (!Character.isJavaIdentifierPart(cCharAt)) {
                    bReturn = true;
                    break;
                }
            }
        }
    }
    } else {
        keys = StringUtils.defaultIfBlank(keys, DEFAULT_SPECIAL_KEY);
        str = StringUtils.defaultIfBlank(str, "");

        if (null != keys) {
            int keysLength = keys.length();
            for (int i = 0; i < keysLength; i++) {
                if (str.indexOf(keys.charAt(i)) > -1) {
                    bReturn = true;
                    break;
                }
            }
        }
    }
    return bReturn;
}

/**
* 카드번호 중간8자리 *표처리.
* <p>
* <pre>
* "1234-****-****-1234" = formatCardNoWithAsterisk("1234567856781234");
* "1234-****-****-1234" = formatCardNoWithAsterisk("1234-5678-5678-1234");
* </pre>
* @param cardNo 카드번호(16자리)
* @return String 카드번호 (1111-****-****-1111)
*/
public static String formatCardNoWithAsterisk(String cardNoVal) {
    String cardNo = cardNoVal;

    cardNo = StringUtils.defaultIfBlank(cardNo, "");
    cardNo = cardNo.replaceAll("[ -]", "");

    if (cardNo.length() == 16) {
        cardNo = cardNo.substring(0, 4) + "-****-****-" + cardNo.substring(12, 16);
    }
    return cardNo;
}

```

```

/**
 * 유효기간 *표처리.
 * <p>
 * <pre>
 * "20**-*2" = formatCardValidDateWithAsterisk("201302");
 * "20**-*2" = formatCardValidDateWithAsterisk("2013-02");
 * </pre>
 * @param validDate 유효기간(6자리)
 * @return String 유효기간 (11**-1*)
 */
public static String formatCardValidDateWithAsterisk(String validDateVal) {
    String validDate = validDateVal;
    validDate = StringUtils.defaultIfBlank(validDate, "");
    validDate = validDate.replaceAll("-", "");

    if (validDate.length() == 6) {
        validDate = validDate.substring(0, 2) + "***-**"; // 앞 2 자리 (년) 노출, 뒷 4 자리
(월) 마스킹 → 20**-**
    }
    return validDate;
}

/**
 * 전화번호의 마지막 번호를 마스킹.
 * <p>
 * 뒷 4자리 마스킹 처리<br>
 * 입력값이 형식에 안 맞으면 "" 값을 리턴한다.
 * <pre>
 * ex) 010-1111-2222 -> 010-1111-****
 * </pre>
 * @param phoneNumber
 * @return String
 */
public static String formatPhoneNumberWithAsterisk(String phoneNumber) {
    if (StringUtils.isBlank(phoneNumber) || phoneNumber.length() <= 4) {
        return "";
    }
    int index = phoneNumber.length();
    return phoneNumber.substring(0, index - 4) + "****";
}

/**
 * email 주소를 마스킹.
 * <p>
 * 앞 4자리 마스킹 처리<br>
 * 입력값이 형식에 안 맞으면 "" 값을 리턴한다.
 * <pre>
 * ex) aaabbcc@nonghyup.com -> ****bcc@nonghyup.com
 * </pre>
 * @param email
 * @return String
 */
public static String formatEmailWithAsterisk(String email) {
    if (StringUtils.isBlank(email)) {
        return "";
    }
    int index = email.indexOf('@');
    if (index < 1) {
        return "";
    }
    if (index > 4) {

```

```

        return StringUtils.repeat("*", 4) + email.substring(4);
    } else {
        return StringUtils.repeat("*", index) + email.substring(index);
    }
}

/**
 * 인터넷 주소 항목을 마스킹.
 * <p>
 * ipV4 : 17~24 비트 영역<br>
 * ipV6 : 113~128 비트 영역<br>
 * 입력값이 형식에 안 맞으면 "" 값을 리턴한다.
 * <pre>
 * ex) 123.123.123.123 -> 123.123.***.123
 *      21DA:00D3:0000:2F3B:02AA:00FF:FE28:9C5A ->
21DA:00D3:0000:2F3B:02AA:00FF:FE28:****
 * </pre>
 * @param ip
 * @return String
 */
public static String formatIPWithAsterisk(String ipVal) {
    String ip = ipVal;
    if (StringUtils.isBlank(ip)) {
        return "";
    }
    if (ip.indexOf('.') > -1) { //ip4
        String temp[] = ip.split("[.]");
        if (temp.length == 4) {
            temp[2] = "****";
            return StringUtils.join(temp, ".");
        } else {
            return "";
        }
    }
    } else if (ip.indexOf(':') > -1) { //ip6
        String temp[] = ip.split("[:]");
        if (temp.length == 8) {
            temp[7] = "****";
            return StringUtils.join(temp, ":");
        } else {
            return "";
        }
    }
    }
    return "";
}

/**
 * UTF-8 인코딩으로 byte 수에 맞춰 문자열 잘라내는 메소드
 * 한글이 바이트 수에 걸쳐서 깨지는 경우 해당 한글 문자를 생략한다.
 */
public static String cutWordUtf8(String wordVal, int byteLength) {
    String word = wordVal;
    try {
        double max_utf8_char_length = 4.0;
        if (word.length() > byteLength) {
            word = word.substring(0, byteLength);
        }
        if (word.length() > byteLength / max_utf8_char_length) {
            int residual = word.getBytes("UTF-8").length - byteLength;
            if (residual > 0) {
                int tempResidual = residual, start, end = word.length();
                while (tempResidual > 0) {

```

```

        start = end - ((int)Math.ceil((double)tempResidual /
max_utf8_char_length));
        tempResidual = tempResidual - word.substring(start,
end).getBytes("UTF-8").length;
        end = start;
    }
    word = word.substring(0, end);
}
}
return word;
} catch (UnsupportedEncodingException e) {
    if (LOGGER.isWarnEnabled()) {
        LOGGER.warn(e.getMessage());
    }
}
return word;
}

/**
 * @description 현재 구동중인 서버가 Paas 환경인지 반환
 * @method isPaas
 * @author CS527980
 * @return
 * @date 2022. 7. 27.
 */
public static boolean isPaas() {
    boolean paas = false;
    if (System.getenv().containsKey("INSTANCE_GUID")) {
        paas = true;
    } else {
        paas = false;
    }
    return paas;
}

public static void getInform(HttpServletRequest request) throws IOException {
    if (LOGGER.isDebugEnabled()) {
        LOGGER.debug("=====getInform(HttpServletRequest
request)=====");
        LOGGER.debug("-----header-----");
        Enumeration<String> headers = request.getHeaderNames();
        if (headers != null) {
            while (headers.hasMoreElements()) {
                String header = headers.nextElement();
                LOGGER.debug("header {} : {}", header, request.getHeader(header));
            }
        }
        LOGGER.debug("-----attribute-----");
        Enumeration<String> attributes = request.getAttributeNames();
        if (attributes != null) {
            while (attributes.hasMoreElements()) {
                String attribute = attributes.nextElement();
                LOGGER.debug("attribute {} : {}", attribute,
request.getAttribute(attribute));
            }
        }
        LOGGER.debug("-----parameter-----");
    }
}

```

```

        Enumeration<String> parameters = request.getParameterNames();
        if (parameters != null) {
            while (parameters.hasMoreElements()) {
                String parameter = parameters.nextElement();
                LOGGER.debug("parameter {} : {}", parameter,
request.getParameter(parameter));
            }
        }
        LOGGER.debug("-----body-----");
        BufferedReader bufferedReader = request.getReader();
        String line = "";
        while ((line = bufferedReader.readLine()) != null) {
            LOGGER.debug(line);
        }
    }

    /**
     * @description IP주소 반환
     * @method getRemoteIp
     * @author CS530226
     * @param request
     * @return sourceIp
     * @date 2022. 7. 27.
     */
    public static String getRemoteIp(HttpServletRequest request) {
        String sourceIp = request.getHeader("X-Forwarded-For");

        //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) :: X-Forwarded-For :: {}
", sourceIp);

        if (sourceIp == null || sourceIp.length() == 0 ||
"unknown".equalsIgnoreCase(sourceIp)) {
            sourceIp = request.getHeader("Proxy-Client-IP");
            //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) :: Proxy-Client-
IP :: {} ", sourceIp);
        }
        if (sourceIp == null || sourceIp.length() == 0 ||
"unknown".equalsIgnoreCase(sourceIp)) {
            sourceIp = request.getHeader("WL-Proxy-Client-IP");
            //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) :: WL-Proxy-Client-
IP :: {} ", sourceIp);
        }
        if (sourceIp == null || sourceIp.length() == 0 ||
"unknown".equalsIgnoreCase(sourceIp)) {
            sourceIp = request.getHeader("HTTP_CLIENT_IP");
            //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) :: HTTP_CLIENT_IP ::
{} ", sourceIp);
        }
        if (sourceIp == null || sourceIp.length() == 0 ||
"unknown".equalsIgnoreCase(sourceIp)) {
            sourceIp = request.getHeader("HTTP_X_FORWARDED_FOR");
            //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) ::
HTTP_X_FORWARDED_FOR :: {} ", sourceIp);
        }
        //위에 언급한 헤더에 모두 없는 경우 getRemoteAddr() 를 통해 ip 추출
        if (sourceIp == null || sourceIp.length() == 0 ||
"unknown".equalsIgnoreCase(sourceIp)) {
            sourceIp = request.getRemoteAddr();
            //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) ::

```

```

HttpServletRequest.getRemoteAddr() :: {} ", sourceIp);
    }

    if (sourceIp.indexOf(",") > -1) {
        sourceIp = sourceIp.split(",")[0];
    }

    //if (logger.isDebugEnabled()) logger.debug("(getRemoteIp) :: return IP :: {} ",
sourceIp);
    return sourceIp;
}

public static void clearThreadContext() {
    for (String s : threadContextkeys) {
        ThreadContext.remove(s);
    }
}

public static void initThreadContext() {
    for (String s : threadContextkeys) {
        ThreadContext.put(s, threadContextDefaultValue);
    }
}

public static Cookie[] getCookieArray(HttpServletRequest request) {
    return request.getCookies();
}

public static String getCookieValue(Cookie cookie) {
    return cookie.getValue();
}

/**
 * 전화번호의 중간 번호를 마스킹.
 * <p>
 * 가운데 번호 마스킹 처리<br>
 * 입력값이 형식에 안 맞으면 "" 값을 리턴한다.
 * <pre>
 * ex) 010-1111-2222 -> 010-****-2222
 * </pre>
 * @param phoneNumber
 * @return String
 */
public static String elkFormatPhoneNumberWithAsterisk(String phoneNumber) {

    String maskingPhoneNumber = "";

    if (StringUtils.isBlank(phoneNumber) || phoneNumber.length() < 11) {
        return "";
    }

    try {
        // 핸드폰 번호 형식에 "-" 문자가 포함 될 경우
        if (phoneNumber.indexOf("-") > -1) {
            int firstDashIndex = phoneNumber.indexOf("-") + 1;
            int lastDashIndex = phoneNumber.lastIndexOf("-");
            int midNoLength = phoneNumber.substring(firstDashIndex,
lastDashIndex).length();

            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < midNoLength; i++) {
                sb.append("*");
            }
        }
    }
}

```

```

    }
    maskingPhoneNumber = phoneNumber.substring(0, phoneNumber.indexOf("-") + 1)
+ sb.toString() + phoneNumber.substring(phoneNumber.lastIndexOf("-"));
    } else {
        maskingPhoneNumber = phoneNumber.substring(0, 3) + "*****" +
phoneNumber.substring(7);
    }
    } catch (Exception e) {
        if (LOGGER.isWarnEnabled()) {
            LOGGER.warn("elkFormatPhoneNumberWithAsterisk ERROR : [{}]", phoneNumber);
        }
    }

    return maskingPhoneNumber;
}

```

```

/**
 * 사업자번호, 주민번호 포매팅.
 * <p>
 * <pre>
 * "123456-1234567" = formatNO("1234561234567", false);
 * "123456-*****" = formatNO("1234561234567", true);
 *
 * "123-45-6789" = formatNO("1234567890", false);
 * "123-45-*****" = formatNO("1234567890", true);
 * </pre>
 *
 * @param no 주민번호 또는 사업자 번호
 * @return java.lang.String
 */
public static String elkFormatRlnoWithAsterisk(String rlNo) {
    if (StringUtils.isEmpty(rlNo)) {
        return "";
    }

    String ssn = rlNo;
    if (ssn.indexOf("-") > -1) {
        ssn.replace("-", "");
    }

    return formatNO(ssn, true);
}

```

```

/**
 * 사용자명 마스킹.
 * <p>
 * <pre>
 * ex) 심청 -> *청
 * ex) 홍길동 -> 홍*동
 * ex) 김수한무 -> 김**무
 * </pre>
 *
 * @param 사용자명
 * @return java.lang.String
 */
public static String elkFormatUserNameWithAsterisk(String userName) {
    String maskingValue = "";
    if (StringUtils.isEmpty(userName)) {
        return "";
    }

    try {

```

18